

Лабораторная работа №8

**Элементы криптографии. Шифрование (кодирование)
различных исходных текстов одним ключом**

Сергеев Даниил Олегович

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Ответы на контрольные вопросы	16
4	Выводы	17
	Список литературы	18

Список иллюстраций

2.1	Пробный запуск программы	12
2.2	Кодирование открытых текстов	13
2.3	Получение комбинации двух шифротекстов через XOR	13
2.4	Расшифровка без ключа	13
2.5	Первая часть расшифровки	14
2.6	Вторая часть расшифровки	14
2.7	Третья часть расшифровки	14
2.8	Четвертая часть расшифровки	15
2.9	Пятая часть расшифровки	15

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом. [1]

2 Выполнение лабораторной работы

Воспользуемся программой `gamma`, написанной в предыдущей лабораторной работе, так как она удовлетворяет требованиям работы. Запустим программу для проверки.

```
ls -l
cd ~/testing
./gamma
```

Будем принимать ключ, открытый текст и шифротекст через аргументы программы:

- Опция `-k` : ключ в hex формате
- Опция `-o` : открытый текст в hex формате
- Опция `-e` : шифротекст в hex формате
- Формат: 00 01 02 ... 1F 2F ...

На выход программа выводит введенные данные (открытый текст отображен в символьном и в шестнадцатеричном формате). В зависимости от входных аргументов:

- Если есть ключ и открытый текст (`new_text, test_decrypt`): выводится шифротекст, полученный путем применения сложения по модулю ключа и текста;

- Если есть шифротекст и ключ (decrypt): выводится расшифровка шифротекста (в байтах и в строчном виде);
- Если есть шифротекст и открытый текст (desired_key): выводится ключ, необходимый для преобразования шифротекста в приведенный открытый текст.

Листинг №1. Код программы

```
#include <stdio.h>
#include <string.h>
#include <locale.h>
#include <stdbool.h>

#define MAX_BUFFER_SIZE 40
typedef unsigned char byte;

void char_2_hex(const char c, byte *val)
{
    if ((c >= '0') && (c <= '9'))
        *val = (byte)(c - '0');
    else if ((c >= 'a') && (c <= 'f'))
        *val = (byte)(c - 'a') + 10;
    else if ((c >= 'A') && (c <= 'F'))
        *val = (byte)(c - 'A') + 10;
}

void read_bytes(byte *buffer, char *data)
{
```

```

    int i = 0, j = 0;
    while (data[i] != '\0')
    {
        byte left = 0;
        byte right = 0;

        char_2_hex(data[i], &left);
        char_2_hex(data[i + 1], &right);

        buffer[j] = (left << 4) | (right & 0x0f);
        if (data[i + 2] == '\0') break;
        i += 3; j++;
    }
}

void print_as_bytes(byte *buffer)
{
    for (int i = 0; i < MAX_BUFFER_SIZE; i++)
    {
        printf("%02X ", buffer[i]);
    }
    printf("\n");
}

void print_as_chars(byte *buffer)
{
    for (int i = 0; i < MAX_BUFFER_SIZE; i++)

```



```

    {
        printf("%c", buffer[i]);
    }
    printf("\n");
}

void xor(byte *buffer, byte *open, byte *key) {
    for (int i = 0; i < MAX_BUFFER_SIZE; i++)
    {
        buffer[i] = open[i] ^ key[i];
    }
}

int main(int argc, char **argv)
{
    byte      key[MAX_BUFFER_SIZE] = { 0 };
    byte      open[MAX_BUFFER_SIZE] = { 0 };
    byte encrypted[MAX_BUFFER_SIZE] = { 0 };

    bool hasKey = false;
    bool hasOpen = false;
    bool hasEncrypted = false;

    int i = 1;
    while (i < argc)
    {
        if (strcmp(argv[i], "-k") == 0) {
            if (i + 1 < argc)

```

```

        {
            read_bytes(key, argv[i + 1]);
            hasKey = true;
        }
    }
    else if (strcmp(argv[i], "-o") == 0) {
        if (i + 1 < argc)
        {
            read_bytes(open, argv[i + 1]);
            hasOpen = true;
        }
    }
    else if (strcmp(argv[i], "-e") == 0) {
        if (i + 1 < argc)
        {
            read_bytes(encrypted, argv[i + 1]);
            hasEncrypted = true;
        }
    }
    i++;
}

printf("key:      (byte)");
print_as_bytes(key); printf("\n");
printf("open:     (byte)");
print_as_bytes(open); printf("\n");
printf("          (char)");
print_as_chars(open); printf("\n");

```

```

printf("encrypted: (byte)");
print_as_bytes(encrypted); printf("\n");

byte buffer[MAX_BUFFER_SIZE] = { 0 };
if (hasOpen && hasKey) {
    xor(buffer, open, key);
    printf("new_text:      (byte)");
    print_as_bytes(buffer); printf("\n");
    xor(buffer, buffer, key);
    printf("test_decrypt: (byte)");
    print_as_bytes(buffer); printf("\n");
    printf("                (char)");
    print_as_chars(buffer); printf("\n");
}
if (hasEncrypted && hasKey) {
    xor(buffer, encrypted, key);
    printf("decrypt:      (byte)");
    print_as_bytes(buffer); printf("\n");
    printf("                (char)");
    print_as_chars(buffer); printf("\n");
}
if (hasEncrypted && hasOpen) {
    xor(buffer, encrypted, open);
    printf("desired_key:  (byte)");
    print_as_bytes(buffer); printf("\n");
}
}

```


[illegible]

Рисунок 2.2: Кодирование открытых текстов

Теперь сложим оба шифротекста по модулю 2. Получим комбинацию двух шифровок, равносильную комбинации двух открытых текстов.

[illegible]

Рисунок 2.3: Получение комбинации двух шифротекстов через XOR

Не зная ключа, прочитаем оба текста. Допустим, нам известно одно сообщение
шаблон: NaVasisxodasiot1204. Используем его в качестве ключа для
зашифрованного текста, полученного ранее.

[illegible]

Рисунок 2.4: Расшифровка без ключа

Применив раскрытое сообщение к шифротексту, получим сообщение-шаблон

Пусть мы знаем часть шаблона: ot1024. Подставив её к шифротексту, мы получим lBanka. Можно предположить, что перед словом Банк стоит слово Филиал - подставим и проверим: odasiiot1204. Так как мы получили осмысленный текст, предположение оказалось верным. Продолжая цепочку рассуждений мы придем к разгадке и расшифруем всё сообщение.

Рисунок 2.5: Первая часть расшифровки

Рисунок 2.6: Вторая часть расшифровки

Рисунок 2.7: Третья часть расшифровки

[illegible]

Рисунок 2.8: Четвертая часть расшифровки

```
[doserge@vdsoserge testing]$ ./gamma -e "18 32 33 17 16 1B ID 11 06 02 08 IF 00 08 03 36 50 5C 5B 55" -k "AE 61 56 61 73 69 73 78 6F 64 61 73 69  
06 F7 31 32 38 34"  
key:      (byte) AE 61 56 61 73 69 73 78 6F 64 61 73 69 69 6F 74 31 32 3D 3A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
  
open:     (byte) 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
  
          (char)  
  
encrypted: (byte) 08 32 33 17 16 1B ID 11 06 02 08 IF 00 08 03 36 50 5C 5B 55 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
  
decrypt:   (byte) 56 53 65 76 65 72 6E 69 69 66 69 6C 69 61 6C 42 61 6E 6E 61 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
  
          (char) VsevernifilialBanka
```

Рисунок 2.9: Пятая часть расшифровки

3 Ответы на контрольные вопросы

1. Как, зная один из текстов (P1 или P2), определить другой, не зная при этом ключа?
 - Зная один открытый текст и оба шифротекста, можно вычислить второй открытый текст через XOR: $P2 = C1 \text{ XOR } C2 \text{ XOR } P1$, поскольку при повторном использовании ключа K справедливо $C1 = P1 \text{ XOR } K$, $C2 = P2 \text{ XOR } K$.
2. Что будет при повторном использовании ключа при шифровании текста?
 - При повторном использовании ключа на шифротексте, мы его расшифруем.
3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов?
 - Реализуется путём наложения одной и той же гаммы на каждый открытый текст отдельно, формируя два шифротекста.
4. Перечислите недостатки шифрования одним ключом двух открытых текстов.
 - Потеря информационной стойкости;
 - Возможность восстановления обоих текстов.
5. Перечислите преимущества шифрования одним ключом двух открытых текстов.
 - Упрощенное управления ключами;

4 Выводы

В результате выполнения лабораторной работы я на практике узнал все преимущества и недостатки однократного гаммирования на примере кодирования различных исходных текстов одним ключом

Список литературы

1. Кулябов Д. С., Королькова А. В., Геворкян М. Н. Основы информационной безопасности (09.03.03) : Лабораторная работа №8. — URL: https://esystem.rudn.ru/pluginfile.php/3096816/mod_resource/content/2/008-lab_crypto-key.pdf (дата обр. 30.05.2026).